

Sistema de autenticación de usuarios

Documento del diseño de la arquitectura

Juan Felipe Rodríguez Amador

August 12, 2026

Contents

1	Descripción del Sistema	2
2	Atributos de Calidad	2
3	Descripción general de la Arquitectura	2
3.1	Diseño de la Arquitectura	2
4	Niveles de la Arquitectura	2
4.1	Nivel de presentación	2
4.2	Nivel de logica de negocio	3
4.3	Nivel de datos	3
5	Tacticas Utilizadas	3
5.1	Alta disponibilidad	3
5.1.1	Redundancia	3
5.1.2	Detección de Fallos	4
5.1.3	Recuperación	4
5.2	Despliegue sencillo	4
5.2.1	Contenedores	4
6	Uso de la aplicación	4
6.1	Instalación	4
6.2	Inicio de Sesión	4

1 Descripción del Sistema

El sistema tiene como objetivo proporcionar una forma de autenticación de usuarios mediante una aplicación móvil de Android nativo. El usuario debe poder ingresar sus credenciales y recibir una respuesta que indique si la autenticación fue exitosa.

Además de que debe mantenerse disponible ante fallos de componentes y a su vez debe ser sencilla de desplegar en un entorno distinto. Por esto, la arquitectura se diseña teniendo como prioridad los atributos de calidad de alta disponibilidad y despliegue sencillo.

2 Atributos de Calidad

La arquitectura prioriza los siguientes atributos de calidad:

- Alta disponibilidad (de un 99.99%)
- Despliegue sencillo

3 Descripción general de la Arquitectura

El sistema sigue una arquitectura de 3 niveles (3-tiers) compuesta por un nivel de presentación, un nivel de lógica de negocio/backend y un nivel de datos.

La separación en tres niveles permite aislar las responsabilidades de cada parte del sistema y facilita tanto el mantenimiento como el despliegue de los diferentes componentes.

3.1 Diseño de la Arquitectura

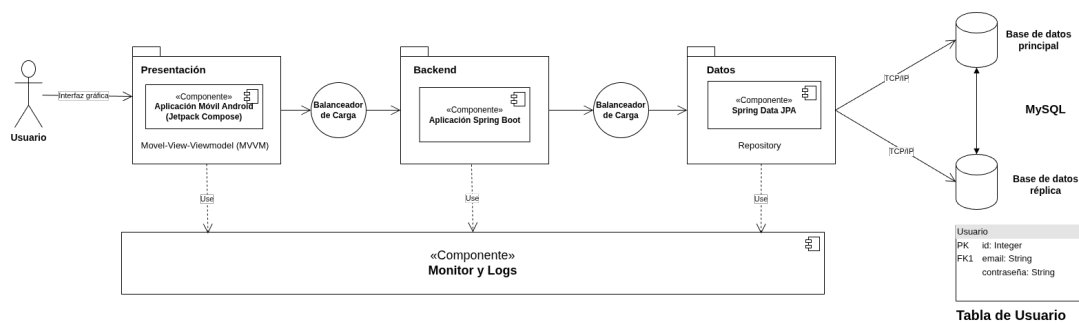


Figure 1: Diagrama lógico de la arquitectura

4 Niveles de la Arquitectura

4.1 Nivel de presentación

El nivel de presentación está compuesto por una aplicación móvil Android desarrollada utilizando Jetpack Compose que sigue el patrón de Model-View-Viewmodel (MVVM).

Su responsabilidad es proporcionar la interfaz mediante la cual el usuario interactúa con el sistema. Siendo esta una pantalla de inicio de sesión y una pantalla principal a la que se accede después de una autenticación exitosa.

La aplicación no implementa directamente la lógica de acceso a la base de datos. En su lugar, envía las credenciales introducidas por el usuario al backend mediante una solicitud HTTP y procesa la respuesta proporcionada por este.

4.2 Nivel de logica de negocio

El nivel de lógica de negocio está compuesto por un balanceador de carga y una instancia de la aplicación Spring Boot.

La aplicación Spring Boot es responsable de procesar las solicitudes de autenticación. Recibe las credenciales proporcionadas por la aplicación móvil, consulta la información correspondiente en la base de datos y determina si las credenciales son válidas.

El balanceador de carga actúa como punto de entrada al backend. Su responsabilidad es distribuir las solicitudes entre las instancias disponibles y evitar enviar solicitudes a una instancia que haya dejado de responder.

4.3 Nivel de datos

El nivel de datos está compuesto por dos instancias de una base de datos MySQL configuradas mediante un esquema de replicación activa-pasiva. Este nivel es responsable de proporcionar almacenamiento persistente para la información utilizada por el sistema de autenticación, principalmente los datos asociados con los usuarios.

En condiciones normales de operación, una de las instancias funciona como base de datos primaria (activa) y es la responsable de atender las operaciones de lectura y escritura realizadas por el backend. La segunda instancia funciona como réplica secundaria (pasiva) y mantiene una copia sincronizada de los datos de la base primaria.

La utilización de dos instancias introduce redundancia en el nivel de datos y permite que el sistema pueda recuperarse ante un fallo de la base de datos primaria.

Se selecciona una configuración activa-pasiva debido a que el sistema de autenticación prioriza la consistencia de la información de los usuarios. Al tener una única instancia como responsable de las operaciones de escritura durante la operación normal, se evita la posibilidad de conflictos de escritura entre ambas bases de datos y se simplifica el mecanismo de recuperación ante fallos.

5 Tacticas Utilizadas

5.1 Alta disponibilidad

Para alcanzar el objetivo de alta disponibilidad se utilizan las siguientes tácticas:

5.1.1 Redundancia

Se despliegan dos instancias de la aplicación Spring Boot y se tiene una replica de la base de datos.

La redundancia evita que el fallo de una única instancia del backend o base de datos provoque necesariamente la interrupción del servicio.

5.1.2 Detección de Fallos

El balanceador de carga verifica que las instancias del backend se encuentren disponibles antes de dirigirles solicitudes.

5.1.3 Recuperación

Cuando una instancia del backend o de la base de datos deja de estar disponible, las solicitudes pueden continuar siendo procesadas por la instancia restante.

5.2 Despliegue sencillo

Para facilitar el despliegue se utiliza la táctica de encapsulamiento mediante contenedores.

5.2.1 Contenedores

Los diferentes servicios necesarios para ejecutar el sistema se empaquetan en contenedores. Esto incluye las instancias del backend, el balanceador de carga y las base de datos.

Los contenedores incluyen las dependencias necesarias para ejecutar cada servicio, reduciendo la cantidad de configuración que debe realizar manualmente el usuario.

6 Uso de la aplicación

6.1 Instalación

Para utilizar la aplicación, se debe instalar el archivo APK proporcionado en un dispositivo Android compatible.

6.2 Inicio de Sesión

Al iniciar la aplicación se mostrará la pantalla de inicio de sesión, el usuario debe introducir:

- * Correo electrónico: correo asociado a una cuenta registrada.
- * Contraseña: contraseña correspondiente a la cuenta.

Después de introducir las credenciales, presionar el botón de inicio de sesión.